



CS RAIDERS

YOUR WAY TO SUCCESS

DATA STRUCTURE QUEUE

Queue

قبل ای حاجه لو مذاکرتش ال stack ارجع ذاكره وافهمه کویس عشان
هیفرق کثیر جدا معاك في درس النهارده

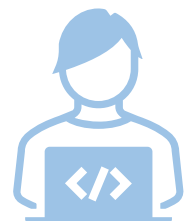
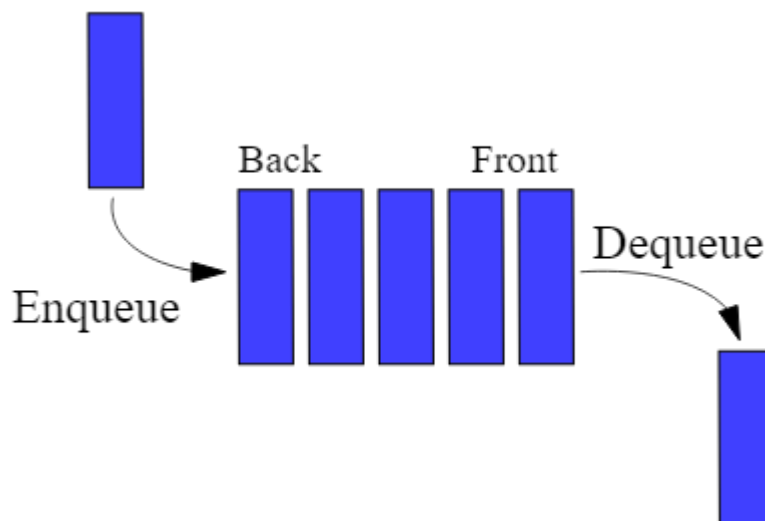
1) what's the meaning of queue ?

Queue is a linear data structure that stores the elements of same type sequentially

2) How these elements sorted ?

It sorted in way that first element enter queue
Will be the first one that leave when dequeue

So , Queue called **“FIFO”** (First In First Out)

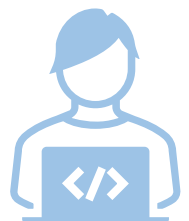


Characteristics of queue :-

- Queue is **Linear - non-primitive** data structure
- Queue is a homogeneous collection of elements.

There're many examples of queue in our life like:-

- Toll station
 - طابور شئون الطلبة
 - Check out in a big super market
-etc



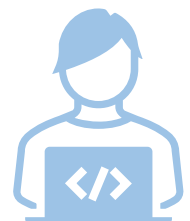
Queue in programming

Queue in programming have 2 indicators

- **Front** → that indicates on first element that will leave queue
- **Rear** → that indicates on last element in Queue

3) Operations of queue

- Create the queue (initialize), leaving it empty.
- Determine whether the queue is empty or not.
- Determine whether the queue is full or not.
- Enqueue (like push in stack) a new entry onto the end of the queue
- Dequeue (like pop in stack) the entry at the front of the queue.

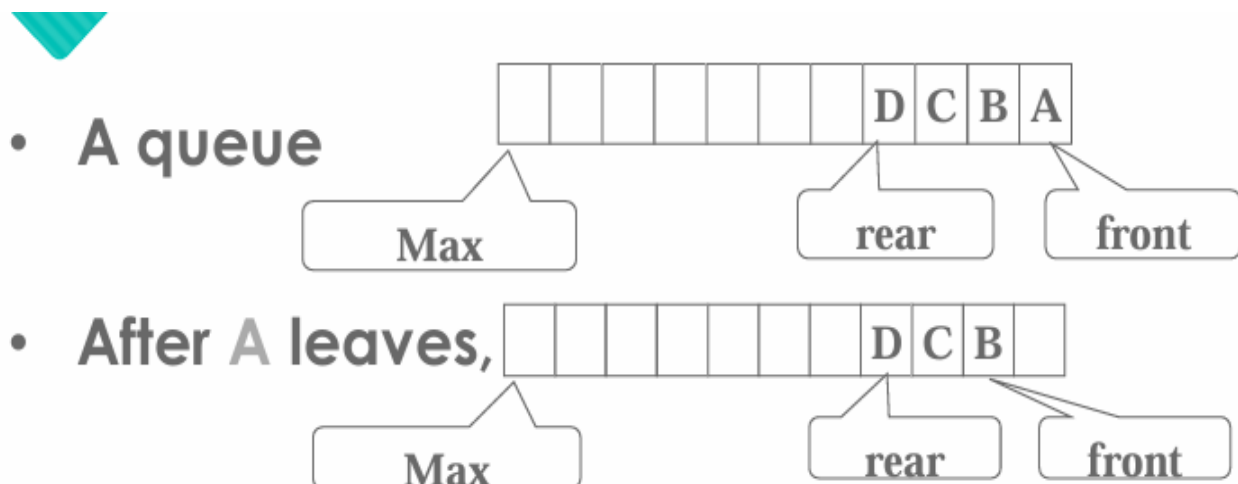


1) initialize queue (create queue)

```
#define    MAX 10
typedef   char EntryType;
typedef   struct {
    int    front;
    int    rear;
    EntryType  entry[MAX];
} QueueType;
```

واضح جدا ان الفرق بين ال queue و stack هو ان هنا عندى rear,front

بس في مشكلة هتقابلنا مع ال linear queue ركز كده هنا



هنا لو جيت سحب B و بعدها C وجيت اضفت عناصر

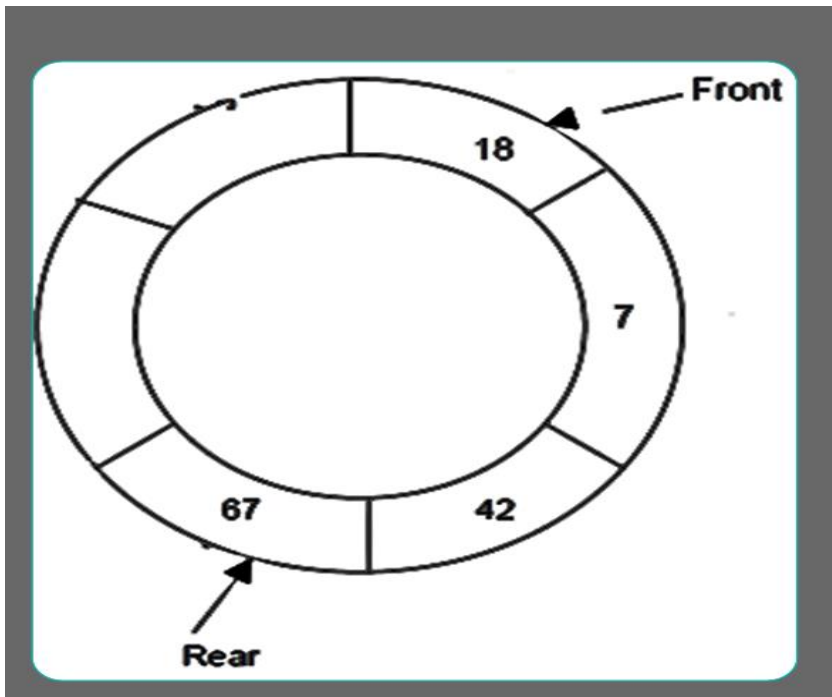
- After B and C leave



لاحظ ان هنا ال queue وصلت لل max مع ان لسه في مكانين فاضيين

فده inefficient

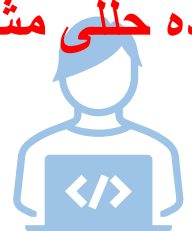
فعشان كده بنلجأ اننا نستعمل ما يسمى بال “Circular Queue”



هنا اخر مكان بيكون مربوط مع اول مكان

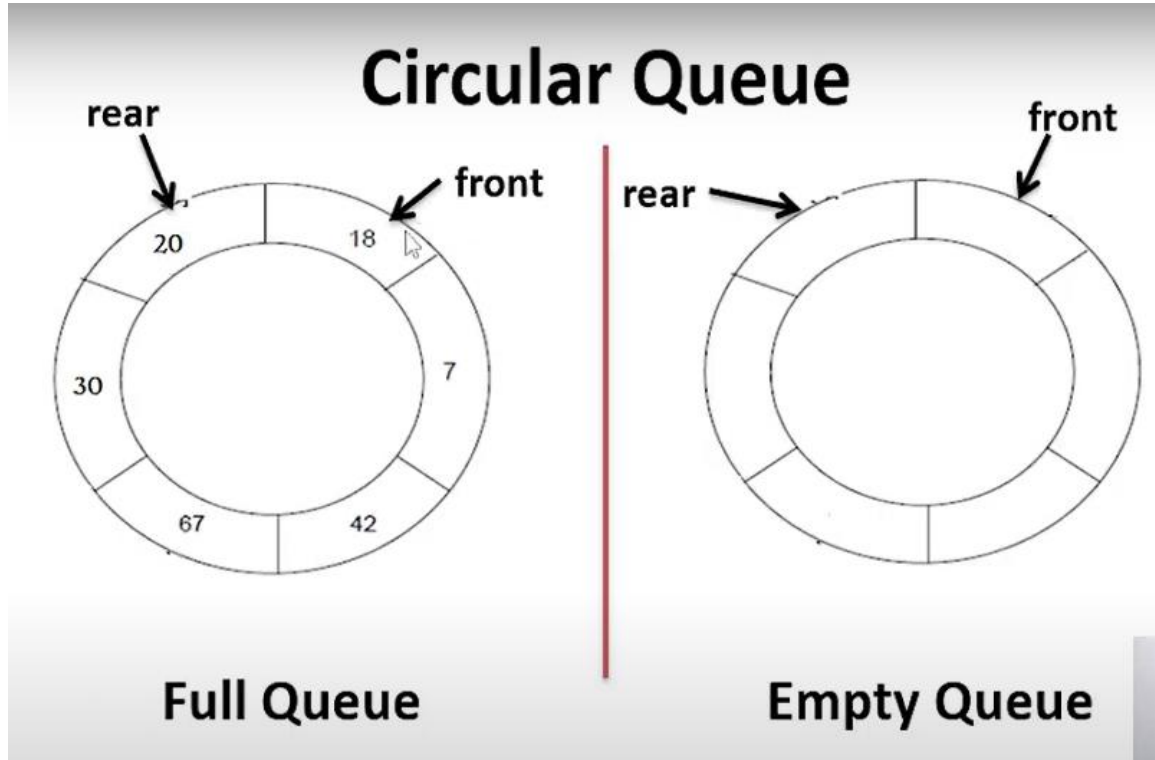
بمعنى انى لو سحب ال 18 في المثال الموضح الفرونت هيبقى 7 عادى و جيت ضفت 3 عناصر هيتضافو واخر واحد فيهم هيكون مكان ال 18 عادى وال rear هيكون مشاور عليه

فده حلى مشكلة ال inefficiency اللى كانت في ال linear queue

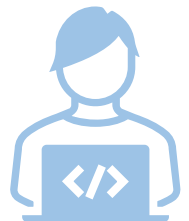


برضو رجعت قابلتنا مشكلة

ان سواء ال queue فاضية او مليانة ال front فارق عن ال rear بمكان واحد



فعشان كده هحتاج حاجة تانية تعرفنى ال size بتاع ال queue
فهنا هلجأ انى اعمل **variable** اسمه **size** من خلاله اقدر اعرف



1) initialization

```
#define queueEntry int
#define maxsize 5

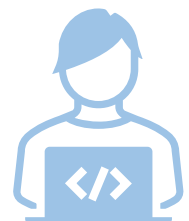
typedef struct queue{
    int front ,rear,size;
    queueEntry arr[maxsize];
}Queue;
```

2) check if queue is empty

```
bool isEmpty(Queue *pq) {
    if(pq->size == 0)
        return true;
    else return false;
}
```

3) check if queue is full

```
bool isFull(Queue *pq) {
    if(pq->size == maxsize)
        return true;
    else return false;
}
```



remid \

$1 \% 10 = 1$

$2 \% 10 = 2$

$X \text{ (when } X < 10) \% 10 = X$

$10 \% 10 = 0$ ولو جيت حسب

4) Enqueue (like push in stack)

```
void Enqueue(queueEntry item, Queue *q){
    if(isFull(q)){
        return ;
    }
```

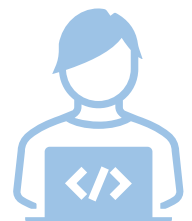
```
q->rear=(q->rear+1) % MAX ;
```

// هنا لما بضيف element جديد بزود ال rear وازوده واحد عشان واحد
فيه ال index الجديد اللي هحط فيه ال element الجديد بشرط لو هو
وصل لل max يبقى بصفر

```
q->entry[q->rear] = item ;
```

```
q-> size ++;
```

```
}
```



3) Dequeue

```
void Dequeue (queueEntry *item, Queue *q){  
    if(isEmpty(q)){  
        return;  
    }  
    *item = q->entry[q->front];  
    q->front =(q->front+1) %MAX ;  
    q-> size --;  
}
```

